

Let me provide a comprehensive explanation of searching and sorting algorithms, focusing on both theoretical understanding and practical implementation.

LECTURE PART

1. Objectives of Searching

Searching is a fundamental operation in computer science that aims to find a specific element (called the key) within a collection of data. The main objectives are to:

- Find elements quickly and efficiently
- Minimize the number of comparisons needed
- Work with both sorted and unsorted data (depending on the algorithm)

2. The Sequential Search

Sequential search (also called linear search) is the simplest searching algorithm. It works by checking each element in sequence until finding the target or reaching the end.

Here's an implementation with detailed analysis:

```
```java
public class SequentialSearch {
 public static int search(int[] array, int target) {
 // Time Complexity: O(n) - must check each element in worst case
 // Space Complexity: O(1) - uses constant extra space

 for (int i = 0; i < array.length; i++) {
 if (array[i] == target) {
 return i; // Found the target, return its index
 }
 }
 return -1; // Target not found
 }
}
```
```

Analysis of Sequential Search:

- Best case: $O(1)$ - target is the first element
- Average case: $O(n/2)$ - target is in the middle
- Worst case: $O(n)$ - target is last or not present
- Works on both sorted and unsorted arrays
- Simple to implement but inefficient for large datasets

3. The Binary Search

Binary search is a more efficient algorithm that works on sorted arrays by repeatedly dividing the search space in half.

```
```java
public class BinarySearch {
 public static int search(int[] sortedArray, int target) {
 // Time Complexity: $O(\log n)$ - divides search space in half each time
 }
}
```

```

// Space Complexity: O(1) - iterative version uses constant space

int left = 0;
int right = sortedArray.length - 1;

while (left <= right) {
 int mid = left + (right - left) / 2; // Avoid potential overflow

 if (sortedArray[mid] == target) {
 return mid; // Found the target
 }

 if (sortedArray[mid] < target) {
 left = mid + 1; // Target is in right half
 } else {
 right = mid - 1; // Target is in left half
 }
}
return -1; // Target not found
}
}
...

```

#### Analysis of Binary Search:

- Time complexity:  $O(\log n)$  in all cases
- Requires sorted input array
- Very efficient for large datasets
- More complex to implement than sequential search

## 4. Introduction to Sorting

Sorting algorithms arrange elements in a specific order (usually ascending or descending). Let's examine each algorithm:

### A. Selection Sort

Works by repeatedly finding the minimum element and putting it in the correct position.

```

```java
public class SelectionSort {
    public static void sort(int[] array) {
        // Time Complexity: O(n^2) - two nested loops
        // Space Complexity: O(1) - sorts in-place

        for (int i = 0; i < array.length - 1; i++) {
            int minIndex = i;

            // Find minimum element in unsorted part
            for (int j = i + 1; j < array.length; j++) {
                if (array[j] < array[minIndex]) {
                    minIndex = j;
                }
            }
        }
    }
}

```

```

        // Swap found minimum with first element of unsorted part
        if (minIndex != i) {
            int temp = array[i];
            array[i] = array[minIndex];
            array[minIndex] = temp;
        }
    }
}
...

```

B. Insertion Sort

Builds the final sorted array one element at a time by repeatedly inserting elements into their correct position.

```

```java
public class InsertionSort {
 public static void sort(int[] array) {
 // Time Complexity: O(n²) in worst case
 // Best Case: O(n) when array is already sorted
 // Space Complexity: O(1) - sorts in-place

 for (int i = 1; i < array.length; i++) {
 int key = array[i];
 int j = i - 1;

 // Move elements greater than key to one position ahead
 while (j >= 0 && array[j] > key) {
 array[j + 1] = array[j];
 j--;
 }
 array[j + 1] = key;
 }
 }
}
...

```

## C. Quick Sort

Uses divide-and-conquer strategy by selecting a 'pivot' element and partitioning the array around it.

```

```java
public class QuickSort {
    public static void sort(int[] array) {
        quickSort(array, 0, array.length - 1);
    }

    private static void quickSort(int[] array, int low, int high) {
        // Time Complexity: O(n log n) average case
        // Space Complexity: O(log n) due to recursion

        if (low < high) {

```

```

        int pivotIndex = partition(array, low, high);

        // Sort elements before and after partition
        quickSort(array, low, pivotIndex - 1);
        quickSort(array, pivotIndex + 1, high);
    }
}

private static int partition(int[] array, int low, int high) {
    int pivot = array[high];
    int i = low - 1; // Index of smaller element

    for (int j = low; j < high; j++) {
        // If current element is smaller than pivot
        if (array[j] < pivot) {
            i++;

            // Swap array[i] and array[j]
            int temp = array[i];
            array[i] = array[j];
            array[j] = temp;
        }
    }

    // Place pivot in correct position
    int temp = array[i + 1];
    array[i + 1] = array[high];
    array[high] = temp;

    return i + 1;
}
}
...

```

LAB IMPLEMENTATION

Here's a complete program implementing all the required searching and sorting algorithms:

```

```java
public class SearchingSortingLab {
 // Sequential Search Implementation
 public static int sequentialSearch(int[] array, int target) {
 System.out.println("Performing Sequential Search for " + target);

 for (int i = 0; i < array.length; i++) {
 System.out.println("Checking index " + i + ": " + array[i]);
 if (array[i] == target) {
 System.out.println("Found at index " + i);
 return i;
 }
 }
 }
}

```

```

 System.out.println("Element not found");
 return -1;
 }

// Binary Search Implementation
public static int binarySearch(int[] sortedArray, int target) {
 System.out.println("Performing Binary Search for " + target);

 int left = 0;
 int right = sortedArray.length - 1;

 while (left <= right) {
 int mid = left + (right - left) / 2;
 System.out.println("Checking mid index " + mid + ": " + sortedArray[mid]);

 if (sortedArray[mid] == target) {
 System.out.println("Found at index " + mid);
 return mid;
 }

 if (sortedArray[mid] < target) {
 System.out.println("Target is in right half");
 left = mid + 1;
 } else {
 System.out.println("Target is in left half");
 right = mid - 1;
 }
 }

 System.out.println("Element not found");
 return -1;
}

// Insertion Sort Implementation
public static void insertionSort(int[] array) {
 System.out.println("Performing Insertion Sort");

 for (int i = 1; i < array.length; i++) {
 int key = array[i];
 int j = i - 1;

 System.out.println("Inserting " + key);

 while (j >= 0 && array[j] > key) {
 array[j + 1] = array[j];
 j--;
 }
 array[j + 1] = key;

 // Print current state of array
 System.out.print("Current array: ");
 printArray(array);
 }
}

```

```

 }
}

// Quick Sort Implementation
public static void quickSort(int[] array) {
 System.out.println("Performing Quick Sort");
 quickSortRecursive(array, 0, array.length - 1);
}

private static void quickSortRecursive(int[] array, int low, int high) {
 if (low < high) {
 System.out.println("Partitioning array from index " + low + " to " + high);
 int pivotIndex = partition(array, low, high);

 quickSortRecursive(array, low, pivotIndex - 1);
 quickSortRecursive(array, pivotIndex + 1, high);
 }
}

private static int partition(int[] array, int low, int high) {
 int pivot = array[high];
 System.out.println("Pivot value: " + pivot);

 int i = low - 1;

 for (int j = low; j < high; j++) {
 if (array[j] < pivot) {
 i++;
 swap(array, i, j);
 }
 }

 swap(array, i + 1, high);
 System.out.print("After partition: ");
 printArray(array);

 return i + 1;
}

private static void swap(int[] array, int i, int j) {
 int temp = array[i];
 array[i] = array[j];
 array[j] = temp;
}

private static void printArray(int[] array) {
 for (int value : array) {
 System.out.print(value + " ");
 }
 System.out.println();
}

```

```

// Main method to test implementations
public static void main(String[] args) {
 int[] testArray = {64, 34, 25, 12, 22, 11, 90};
 System.out.println("Original array:");
 printArray(testArray);

 // Test Sequential Search
 sequentialSearch(testArray, 22);

 // Test Binary Search (needs sorted array)
 int[] sortedArray = {11, 12, 22, 25, 34, 64, 90};
 binarySearch(sortedArray, 25);

 // Test Insertion Sort
 int[] arrayForInsertion = testArray.clone();
 insertionSort(arrayForInsertion);

 // Test Quick Sort
 int[] arrayForQuick = testArray.clone();
 quickSort(arrayForQuick);

 System.out.println("Final sorted array:");
 printArray(arrayForQuick);
}
}
...

```

This implementation includes:

1. Detailed comments explaining each step
2. Print statements to show the progress of each algorithm
3. A main method to test all implementations
4. Error handling and edge cases
5. Visualization of the sorting process

Would you like me to explain any specific part in more detail or provide additional examples of these algorithms in action?